

Assignment 2: Secure Migration of a Traditional Application to AWS (Cloud Security)

Allocated Marks: 10%

Objective

This assignment aims to provide practical experience in securely modernizing and migrating a legacy on-premises application to a secure and resilient AWS cloud architecture. Students will analyze security weaknesses in legacy systems, design cloud-native security controls, implement the solution using Infrastructure as Code (IaC), and validate the deployed environment against common security threats.

Grouping

To be done in a group of three (3) students. Students who cannot form a 3-member group can resort to a 2-member group. Groups with fewer than two members are not allowed.

Report Formatting

It is mandatory to use the MMU assignment front page with group members' details. Font allowed: Times New Roman or Calibri with size 12. All diagrams and figures must be labelled and captioned. **The maximum number of pages allowed is 10.**

The cover page should be like this:



CCS6344 T2530 Assignment 2 Submission

Group Name: Group A

Name 1	1331101423
Name 2	1441101424
Name 3	1551101425

Deadline

Submit your deliveries before **11:59 p.m., Friday, 5th February 2026**. Every member of the group must submit their **report only** individually on eBwise. **Please provide the YouTube demo link in the first page of your report.**

Scenario:

You are part of a **Cloud Security Consulting Team** hired to modernize a legacy, monolithic application currently hosted on a single on-premises server. The server hosts:

- A web application (e.g., Apache/Nginx + PHP, Python, or Node.js)
- A locally installed database (e.g., MySQL, MSSQL)

The application supports a small-scale use case such as:

- Student information management
- Internal employee portal
- Simple inventory or order management system

Your task is to **redesign, migrate, and secure** the application using AWS services while ensuring **confidentiality, integrity, availability, and compliance**.

Deliverables & Questions:

Part A: Legacy Security Risk Assessment (20 Marks)

1. **Identify and explain at least SIX (6) security risks** present in the existing on-premises architecture.

Your analysis must include risks related to:

- Network exposure and firewall limitations.
- Privileged access and account management.
- Software vulnerabilities and patch management.
- Data protection, backups, and disaster recovery.
- Logging, monitoring, and incident detection.

2. **Rank the identified risks** based on severity (High / Medium / Low) and briefly justify your ranking.

Part B: Secure AWS Architecture Design (25 Marks)

3. Design a **secure and highly available AWS architecture** for the migrated application.

Your architecture diagram must clearly show:

- Public and private subnets within a VPC.
- Application tier and database tier separation.
- Load balancing and secure access mechanisms.

4. Your design must include and justify the use of the following AWS services:

- Compute: EC2 *or* ECS/Fargate.
- Database: Amazon RDS (Multi-AZ preferred).
- Networking: VPC, Subnets, Security Groups, NACLs.
- Security: IAM, AWS WAF, CloudTrail, CloudWatch.
- Data Protection: Encryption at rest and in transit.

Part C: Security-Focused Migration Strategy (15 Marks)

5. Propose a **step-by-step secure migration strategy**, explaining:
 - How data will be migrated securely to AWS.
 - How downtime and data loss risks will be minimized.
 - How access control and secrets will be managed.
6. Map each **legacy security risk** identified in Part A to a **specific AWS security control** used to mitigate it.

Part D: Secure Implementation on AWS (30 Marks)

7. Implement the proposed architecture using **Infrastructure as Code (IaC)** (CloudFormation or Terraform)
 - Code must be modular, readable, and well-documented.
8. Deploy the application and demonstrate:
 - Secure IAM roles and least-privilege policies.
 - Encrypted RDS and S3 storage.
 - Restricted Security Group rules.
 - HTTPS configuration using SSL/TLS.
9. Upload all application and **IaC code to GitHub** and include the repository link in the report.

Part E: Security Validation & Reflection (Report & Demo) (10 Marks)

10. Security Testing: Perform basic security validation tests on your deployed application. This could include:
 - Port scanning your public endpoints (from an external machine) to ensure only expected ports are open.
 - Attempting to bypass WAF rules (e.g., simple SQL injection attempts).
 - Verifying CloudTrail logs for security-relevant events.
 - Confirming encryption is enabled for RDS and S3.

Security Report: Document the results of your security validation. Discuss any challenges encountered and how they were overcome.

Demonstration: Be prepared to demonstrate your deployed application and walk through the AWS console to showcase the implemented security controls (Security Groups, IAM roles, WAF, CloudTrail, etc.).

Please create a video presentation of not more than 10 minutes and upload in the YouTube. Please provide the YouTube link in the first page of the report.

Perform **basic security validation**, including at least THREE (3) of the following:

- Port scanning of public endpoints
- Verification of WAF rule enforcement
- Review of CloudTrail logs
- Confirmation of encryption settings

Bonus Marks (Optional)

- Implementation of the design using IPv6 addressing.
- Containerize the application using Docker and deploy via ECS/Fargate.
- Integrate a simple DevSecOps pipeline.
- Use AWS Inspector or similar tools for vulnerability scanning.
- Discuss basic cost-security trade-offs.

Tools & Technologies:

- AWS Management Console.
- AWS CLI.
- AWS CloudFormation or Terraform.
- Basic web application (e.g., PHP/MySQL, Python/Flask/PostgreSQL, Node.js/Express).
- Diagramming tool (draw.io, Lucidchart).
- Basic security testing tools (e.g., nmap, a simple web proxy like Burp Suite Community Edition for manual WAF testing).

Assessment Criteria:

- **Completeness and Accuracy:** All deliverables submitted, and technical implementations are correct.
- **Security Best Practices:** Adherence to AWS security best practices (least privilege, defense-in-depth, encryption, logging).
- **Architectural Design:** Clarity, scalability, and security of the proposed AWS architecture.
- **Infrastructure as Code Quality:** Readability, reusability, and correctness of IaC.
- **Security Validation:** Thoroughness of security testing and analysis of results.
- **Presentation and Communication:** Clear explanation of concepts and demonstration of the solution.

Here are detailed rubrics for the "Secure Migration of a Traditional Application to AWS" assignment, broken down by each deliverable and assessment criteria.

Rubrics: Secure Migration of a Traditional Application to AWS

Total Points: 100

Part A: Legacy Security Risk Assessment (20 Marks)

Criteria	Poor (0–4)	Fair (5–8)	Good (9–12)	Very Good (13–16)	Excellent (17–20)
Identification of Security Risks	Fewer than 3 risks identified; irrelevant or incorrect	3–4 risks identified with limited explanation	5–6 relevant risks identified with basic explanation	6 well-explained risks with clear relevance	6+ comprehensive, well-justified risks

Criteria	Poor (0–4)	Fair (5–8)	Good (9–12)	Very Good (13–16)	Excellent (17–20)
Risk Severity & Justification	No prioritization or incorrect ranking	Basic ranking with weak justification	Logical ranking with acceptable justification	Clear severity ranking with strong reasoning	Insightful prioritization demonstrating deep security understanding

Part B: Secure AWS Architecture Design (25 Marks)

Criteria	Poor (0–5)	Fair (6–10)	Good (11–15)	Very Good (16–20)	Excellent (21–25)
Architecture Diagram	Missing or unclear	Diagram present but incomplete	Clear diagram with core components	Well-structured, secure, scalable design	Professional-quality diagram with layered security
AWS Service Selection	Incorrect or missing services	Limited AWS services used	Correct core services included	All required services correctly used	Optimal service selection with best practices
Security Design	Little or no security consideration	Basic security controls shown	Appropriate security controls included	Strong defense-in-depth approach	Advanced security layering and justification

Part C: Security-Focused Migration Strategy (15 Marks)

Criteria	Poor (0–3)	Fair (4–6)	Good (7–9)	Very Good (10–12)	Excellent (13–15)
Migration Plan	No clear steps	Basic steps, poorly explained	Logical steps with some detail	Clear, structured migration plan	Detailed, secure, and realistic plan
Risk-to-Control Mapping	No mapping	Weak or incomplete mapping	Most risks mapped to controls	Clear mapping for all risks	Insightful, accurate mapping with justification

Part D: Secure AWS Implementation (30 Marks)

Criteria	Poor (0–6)	Fair (7–12)	Good (13–18)	Very Good (19–24)	Excellent (25–30)
Infrastructure as Code (IaC)	Missing or non-functional	Partial deployment	Mostly functional IaC	Fully functional, well-documented	Modular, reusable, best-practice IaC
Application Deployment	Not working	Partially functional	Fully functional with minor issues	Fully functional and secure	Optimized, cloud-native deployment

Criteria	Poor (0–6)	Fair (7–12)	Good (13–18)	Very Good (19–24)	Excellent (25–30)
Security Configuration	Insecure defaults	Basic security enabled	Most security controls applied	All required security controls implemented	Advanced, least-privilege, defense-in-depth

Part E: Security Validation & Reflection (10 Marks)

Criteria	Poor (0–2)	Fair (3–4)	Good (5–6)	Very Good (7–8)	Excellent (9–10)
Security Testing	No testing	Minimal testing	Basic tests conducted	Multiple tests with analysis	Thorough testing with insights
Reflection & Learning	Missing	Very brief	Basic reflection	Clear lessons learned	Critical analysis with recommendations